

Time Series Preprocessing and Univariate Modeling Using Divvy Bike Data

Time series data is common in data science, and effective preprocessing and modeling are essential for extracting meaningful insights and making accurate forecasts. In this lesson, we will walk through the process of aggregating, preparing, and modeling Divvy bikeshare data, focusing on univariate time series prediction. You will learn practical techniques for handling real-world time series and building foundational forecasting models.

Learning Objectives

By the end of this lesson, you will be able to:

- Understand the importance of data preprocessing for time series modeling.
- Merge and clean real-world datasets (Divvy trip data and weather data) for analysis.
- Engineer features relevant to time series forecasting, including lagged variables and rolling means.
- Explore and visualize time series data to identify trends, seasonality, and relationships with external factors.
- Assess and transform time series data for stationarity using statistical tests and transformations.
- Prepare data for model validation by performing appropriate train-test splits for time series.

Dataset Case Study: Bicycle Sharing in Chicago

This case study uses real-world data from the Divvy bikeshare system in Chicago, selected for the [MCDC](#) practicum course (Winter 2022, [Dr. Arend Kuyper](#)) in partnership with [Equiticity](#) and community leader Oboi Reed. The project aimed to address real community needs through data science, with results featured on the [MCDC Past Projects website](#).

Context:

- The Divvy system, operated in collaboration with the Chicago Department of Transportation, provides 6,000 bikes at 570+ stations across Chicago and Evanston.
- The dataset includes daily trip data and weather information, enabling analysis of rider-ship patterns and external influences.
- Students worked in teams to respond to Equiticity's Request for Data Science Services, applying statistical and machine learning methods to real-world questions.

Modeling Focus:

- The primary goal is to forecast daily bike pick-ups using time series methods.
- Tasks include data cleaning, feature engineering, and both classical and machine learning modeling approaches.
- The project demonstrates how data science can inform operations, improve user experience, and support community-driven decision making.

This lesson was co-developed by **Dr. Lizhen Shi** and **Dr. Arend Kuyper** for the MCDC pathway courses (Summer 2025). The dataset is publicly available from the [Divvy System Data Portal](#).

Guided Practice

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

# ignore warnings
import warnings
warnings.filterwarnings('ignore')

#matplotlib inline
plt.style.use('seaborn-v0_8-whitegrid') # Updated seaborn style name
```

Step 1: Data Preparation

Load, merge, and clean Divvy trip and weather data to create a unified dataset for analysis. Ensuring data quality and compatibility is essential for accurate modeling.

```
trips = pd.read_csv('datasets/2021-2023-divvy-tripdata-cleaned.csv')

print(trips.shape)

trips.head()
```

(14191009, 34)

	ride_id	rideable_type	started_at	ended_at	start_station_name
0	B9F73448DFBE0D45	classic_bike	2021-01-24 19:15:38	2021-01-24 19:22:51	California Ave & Cort
1	457C7F4B5D3DA135	electric_bike	2021-01-23 12:57:38	2021-01-23 13:02:10	California Ave & Cort
2	57C750326F9FDABE	electric_bike	2021-01-09 15:28:04	2021-01-09 15:37:51	California Ave & Cort
3	4D518C65E338D070	electric_bike	2021-01-09 15:28:57	2021-01-09 15:37:54	California Ave & Cort
4	9D08A3AFF410474D	classic_bike	2021-01-24 15:56:59	2021-01-24 16:07:08	California Ave & Cort

Adding the weather data

Weather, especially temperature, is a key external factor influencing daily bike ridership. By integrating weather data, we can build more accurate and insightful models that account for these effects. In this lesson, we will merge the weather data with the trip data to enable multivariate time series modeling.

```
weather_data = pd.read_csv('datasets/weather_temp.csv', parse_dates=['date'])
weather_data['date'] = pd.to_datetime(weather_data['date']).dt.date
weather_data.sort_values(by='date', inplace=True)

print(weather_data.shape)
display(weather_data.isnull().sum())
weather_data.head()
```

(1095, 2)

```
date          0
temperature   0
dtype: int64
```

	date	temperature
0	2021-01-01	31.5
1	2021-01-02	33.1
2	2021-01-03	32.9
3	2021-01-04	29.6
4	2021-01-05	31.3

Ensure datetime compatibility before merging datasets

Before merging, make sure both the trip and weather datasets have their date columns in a compatible datetime format. This step is essential for a successful and accurate merge.

```
# convert the started_date to date
trips['started_date'] = pd.to_datetime(trips['started_date'])

# convert the date in weather_data to date
weather_data['date'] = pd.to_datetime(weather_data['date'])
```

Cleaning and Inspecting

```
# merge the combined_data with the weather_data
trips_with_weather = trips.merge(weather_data, left_on='started_date', right_on='date', how=

# drop the redundant date column
trips_with_weather.drop(columns=['date'], inplace=True)

# check the missing values in the temperature column
print(trips_with_weather['temperature'].isnull().sum())

#show the first few rows
trips_with_weather.head()
```

0

	ride_id	rideable_type	started_at	ended_at	start_station_name
0	B9F73448DFBE0D45	classic_bike	2021-01-24 19:15:38	2021-01-24 19:22:51	California Ave & Corto
1	457C7F4B5D3DA135	electric_bike	2021-01-23 12:57:38	2021-01-23 13:02:10	California Ave & Corto
2	57C750326F9FDABE	electric_bike	2021-01-09 15:28:04	2021-01-09 15:37:51	California Ave & Corto

	ride_id	rideable_type	started_at	ended_at	start_station_name
3	4D518C65E338D070	electric_bike	2021-01-09 15:28:57	2021-01-09 15:37:54	California Ave & Cort
4	9D08A3AFF410474D	classic_bike	2021-01-24 15:56:59	2021-01-24 16:07:08	California Ave & Cort

```
trips_with_weather.columns
```

```
Index(['ride_id', 'rideable_type', 'started_at', 'ended_at',
      'start_station_name', 'start_station_id', 'end_station_name',
      'end_station_id', 'start_lat', 'start_lng', 'end_lat', 'end_lng',
      'member_casual', 'started_year', 'started_date', 'started_month',
      'started_day', 'started_dayofweek', 'started_hour', 'started_quarter',
      'ended_year', 'ended_month', 'ended_day', 'ended_dayofweek',
      'ended_hour', 'ended_quarter', 'ended_date', 'is_start_holiday',
      'is_stop_holiday', 'duration', 'duration_min', 'duration_bin',
      'distance', 'distance_bins', 'temperature'],
      dtype='object')
```

After merging, check that the temperature and other weather features are correctly aligned with each trip date. This ensures our dataset is ready for further analysis.

```
daily_temperature = trips_with_weather.groupby('started_date')['is_start_holiday'].mean().reset_index()
daily_temperature.head()
```

	started_date	is_start_holiday
0	2021-01-01	1.0
1	2021-01-02	0.0
2	2021-01-03	0.0
3	2021-01-04	0.0
4	2021-01-05	0.0

```
# create a dataframe with the number of trips per day, the is_start_holiday column, and the temperature
daily_trips = trips_with_weather.groupby('started_date').size().reset_index(name='num_trips')
daily_temperature = trips_with_weather.groupby('started_date')[['temperature', 'is_start_holiday']].mean().reset_index()

# merge the two dataframes
daily_data = daily_trips.merge(daily_temperature, on='started_date', how='left')
```

```
# print the shape of the daily_data
print(daily_data.shape)

# check for missing values
print(daily_data.isnull().sum())

# show the first few rows
daily_data.head()
```

```
(1095, 4)
started_date      0
num_trips         0
temperature       0
is_start_holiday  0
dtype: int64
```

	started_date	num_trips	temperature	is_start_holiday
0	2021-01-01	908	31.5	1.0
1	2021-01-02	2046	33.1	0.0
2	2021-01-03	2158	32.9	0.0
3	2021-01-04	2854	29.6	0.0
4	2021-01-05	3426	31.3	0.0

Saving and Reading

```
# save the daily_data to a csv file
daily_data.to_csv('datasets/daily_data_with_temperature_holiday.csv', index=False)
```

```
# read the daily_data_with_temperature_holiday.csv file
daily_data = pd.read_csv('datasets/daily_data_with_temperature_holiday.csv')
```

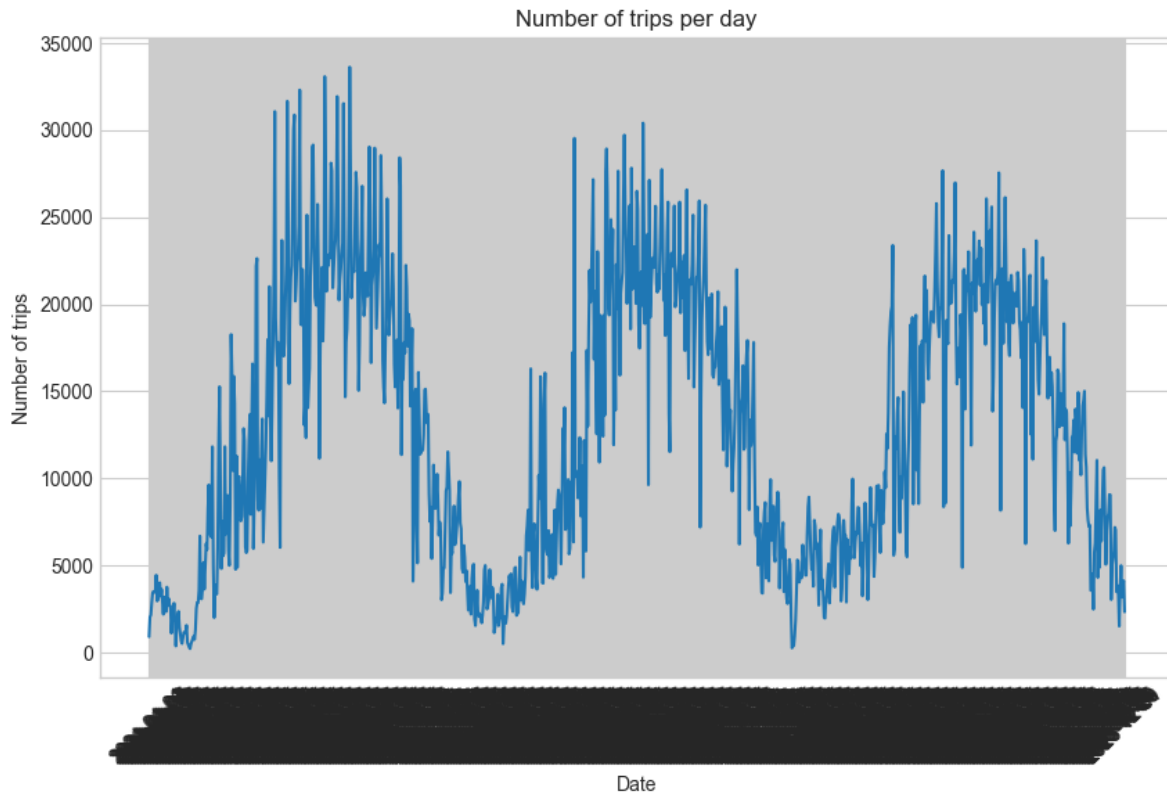
Step 2: Exploratory Data Analysis (EDA)

Examine the distribution of daily trips and explore how ridership relates to temperature. EDA helps uncover patterns and informs feature engineering for modeling.

```
# check out the statistics of the num_trips column
daily_data['num_trips'].describe()
```

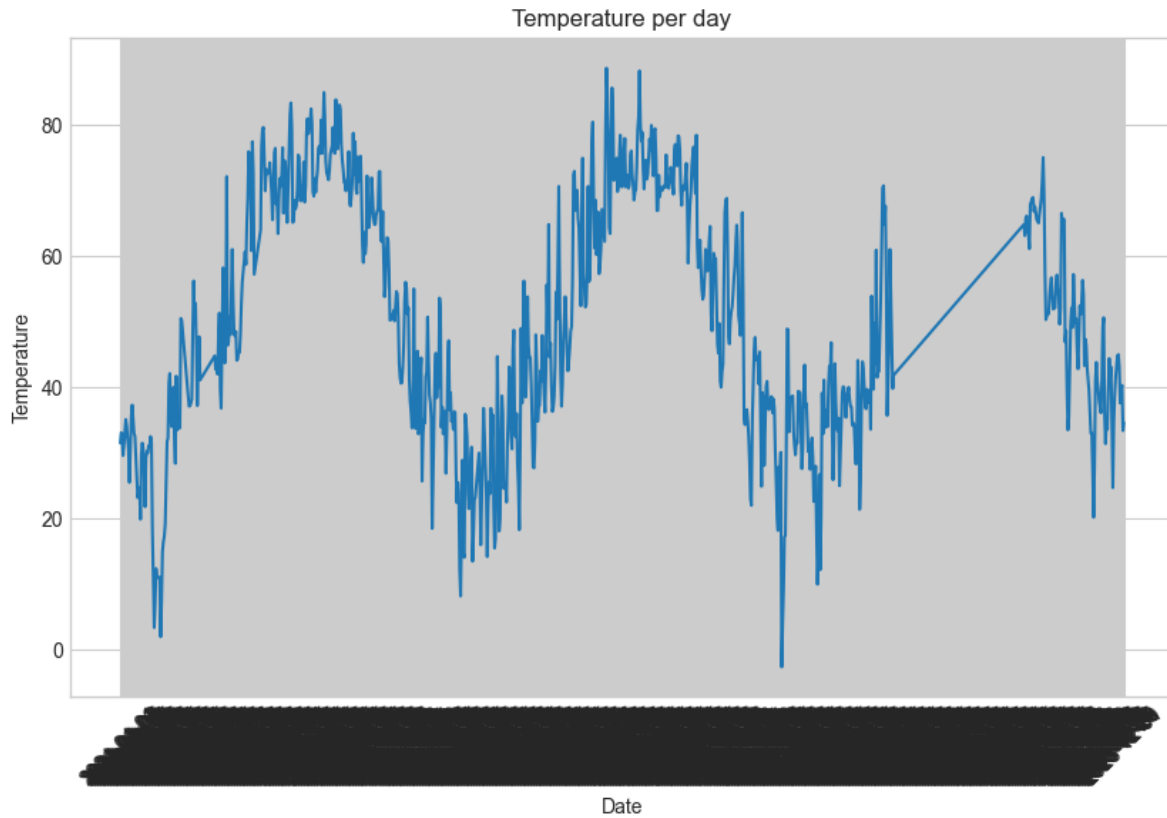
```
count      1095.000000
mean       12959.825571
std        7817.516590
min         219.000000
25%        5900.000000
50%       12335.000000
75%       19896.500000
max       33617.000000
Name: num_trips, dtype: float64
```

```
# plot the number of trips per day
plt.figure(figsize=(10, 6))
sns.lineplot(x='started_date', y='num_trips', data=daily_data)
plt.title('Number of trips per day')
plt.xlabel('Date')
plt.ylabel('Number of trips')
plt.xticks(rotation=45)
plt.show()
```



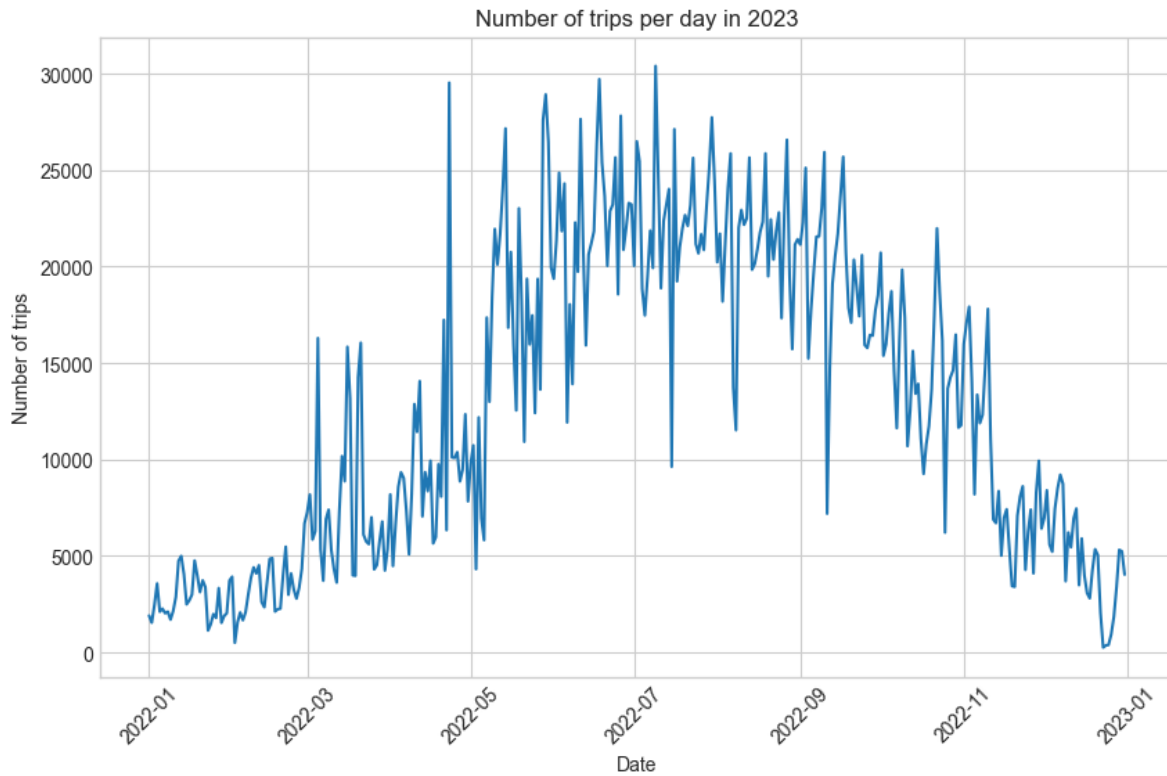
Checking how the number of trips varies with the daily temperatures

```
# plot the temperature per day
plt.figure(figsize=(10, 6))
sns.lineplot(x='started_date', y='temperature', data=daily_data)
plt.title('Temperature per day')
plt.xlabel('Date')
plt.ylabel('Temperature')
plt.xticks(rotation=45)
plt.show()
```

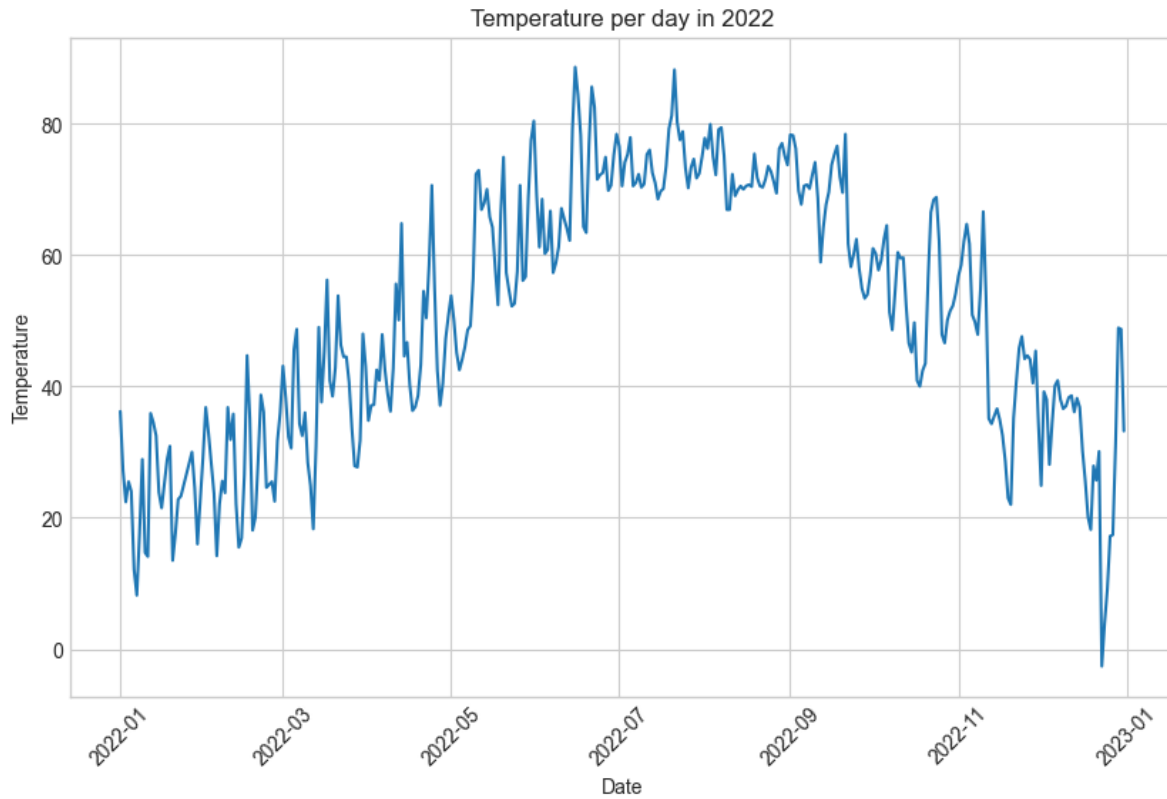


```
# zoom in the year 2022
daily_data['started_date'] = pd.to_datetime(daily_data['started_date'])
daily_data_2022 = daily_data[daily_data['started_date'].dt.year == 2022]

# plot the number of trips per day in 2023
plt.figure(figsize=(10, 6))
sns.lineplot(x='started_date', y='num_trips', data=daily_data_2022)
plt.title('Number of trips per day in 2023')
plt.xlabel('Date')
plt.ylabel('Number of trips')
plt.xticks(rotation=45)
plt.show()
```

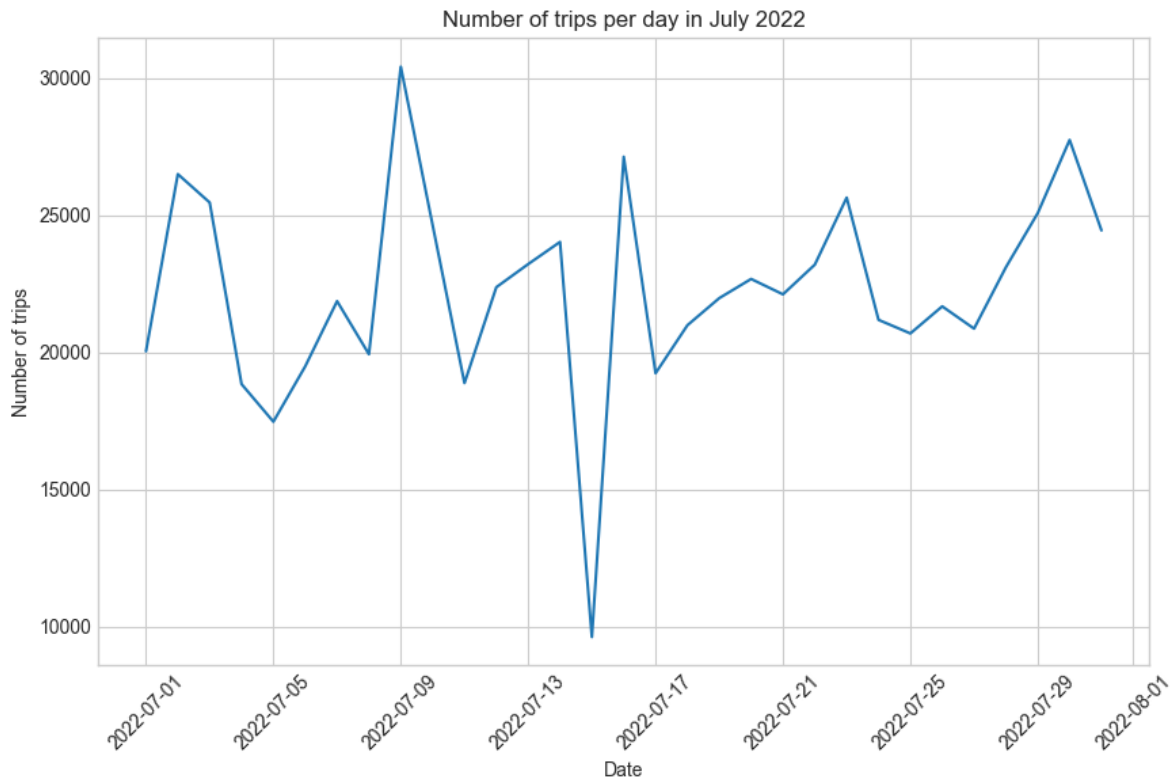


```
# plot the temperature per day in 2023
plt.figure(figsize=(10, 6))
sns.lineplot(x='started_date', y='temperature', data=daily_data_2022)
plt.title('Temperature per day in 2022')
plt.xlabel('Date')
plt.ylabel('Temperature')
plt.xticks(rotation=45)
plt.show()
```

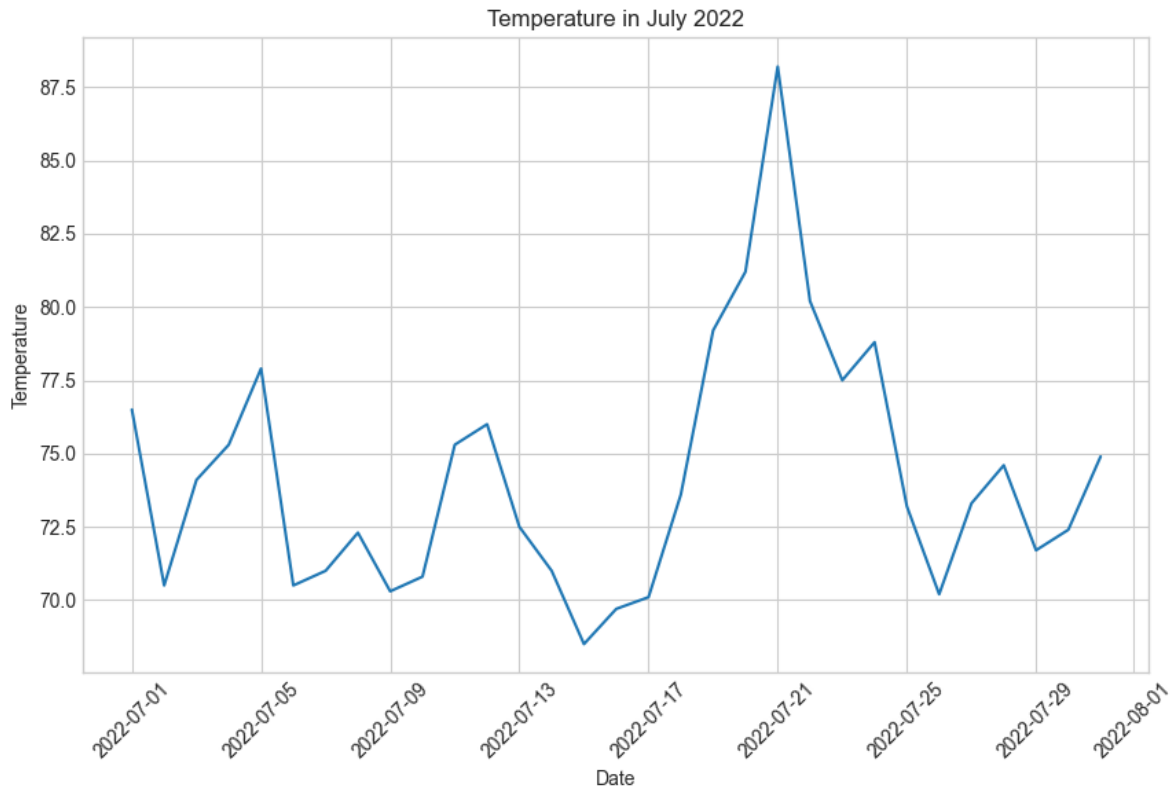


```
# extract the instances which year is 2022, which month is 7
daily_data_2022_7 = daily_data[(daily_data['started_date'].dt.year == 2022) & (daily_data['s

# plot the number of trips per day in July 2022
plt.figure(figsize=(10, 6))
sns.lineplot(x='started_date', y='num_trips', data=daily_data_2022_7)
plt.title('Number of trips per day in July 2022')
plt.xlabel('Date')
plt.ylabel('Number of trips')
plt.xticks(rotation=45)
plt.show()
```



```
# plot the temperature in July 2022
plt.figure(figsize=(10, 6))
sns.lineplot(x='started_date', y='temperature', data=daily_data_2022_7)
plt.title('Temperature in July 2022')
plt.xlabel('Date')
plt.ylabel('Temperature')
plt.xticks(rotation=45)
plt.show()
```



Step 3: Seasonality Analysis

Analyze recurring patterns in the data, such as weekly and yearly cycles. Understanding seasonality helps select appropriate models and improves forecast accuracy. Use decomposition to separate trend, seasonality, and residuals.

```
# First, ensure you have the required libraries
from statsmodels.tsa.seasonal import seasonal_decompose

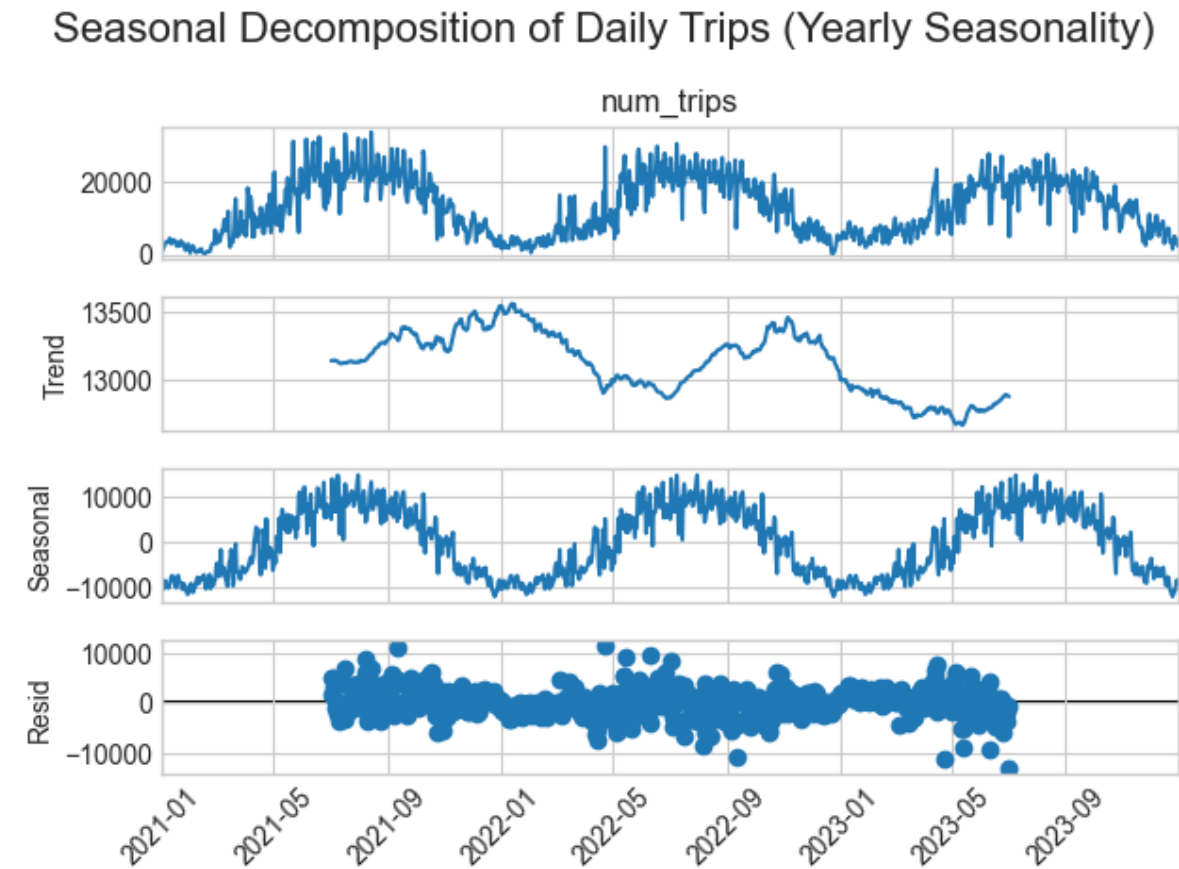
# set the date as the index for meaningful x-axis labels
daily_data_indexed = daily_data.copy()
daily_data_indexed['started_date'] = pd.to_datetime(daily_data_indexed['started_date'])
daily_data_indexed.set_index('started_date', inplace=True)

# do seasonal decomposition using the indexed data
result = seasonal_decompose(daily_data_indexed['num_trips'], model='additive', period=365)

# plot the decomposition with meaningful x-axis
```

```
plt.figure(figsize=(14, 8))
result.plot()
plt.suptitle('Seasonal Decomposition of Daily Trips (Yearly Seasonality)', fontsize=16)
plt.xticks(rotation=45)
plt.tight_layout()
```

<Figure size 1400x800 with 0 Axes>



Weekly Seasonality

Zooming in on 2022, we decompose the data with a weekly period (7 days) to highlight weekly usage patterns. This helps identify short-term cycles, such as higher ridership on weekends.

```

# do seasonal decomposition for the year 2022
daily_data_2022_indexed = daily_data_2022.copy()
daily_data_2022_indexed['started_date'] = pd.to_datetime(daily_data_2022_indexed['started_date'])
daily_data_2022_indexed.set_index('started_date', inplace=True)

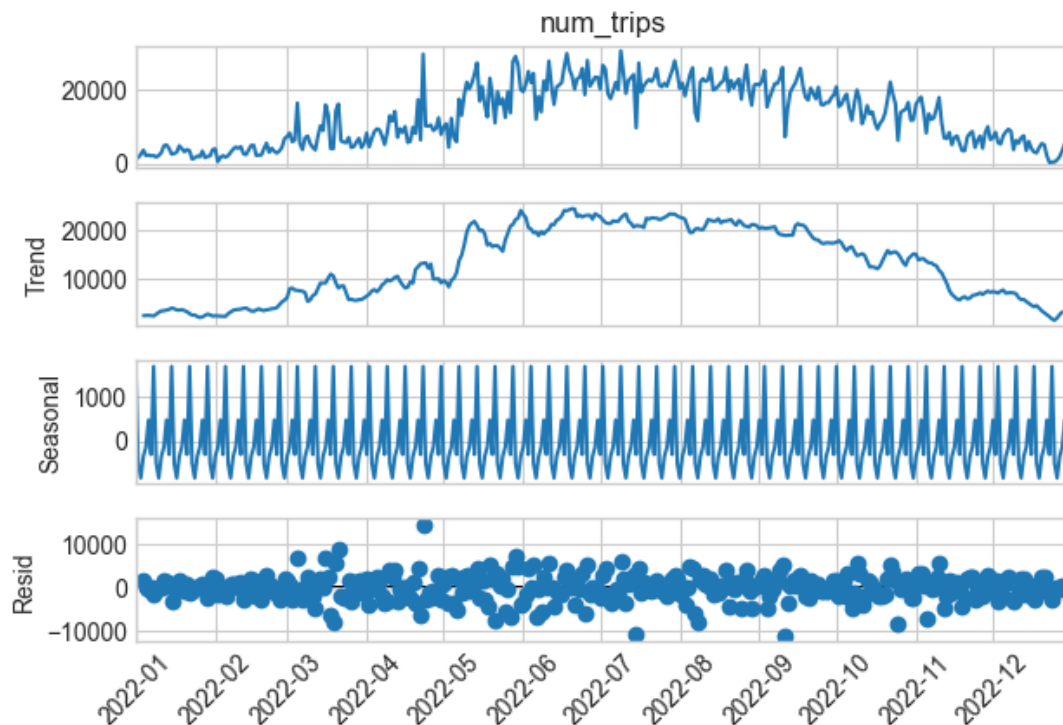
result_2022 = seasonal_decompose(daily_data_2022_indexed['num_trips'], model='additive', per

# plot the decomposition with proper date formatting
plt.figure(figsize=(14, 8))
result_2022.plot()
plt.suptitle('Seasonal Decomposition of Daily Trips - 2022 (Weekly Seasonality)', fontsize=14)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```

<Figure size 1400x800 with 0 Axes>

Seasonal Decomposition of Daily Trips - 2022 (Weekly Seasonality)



```

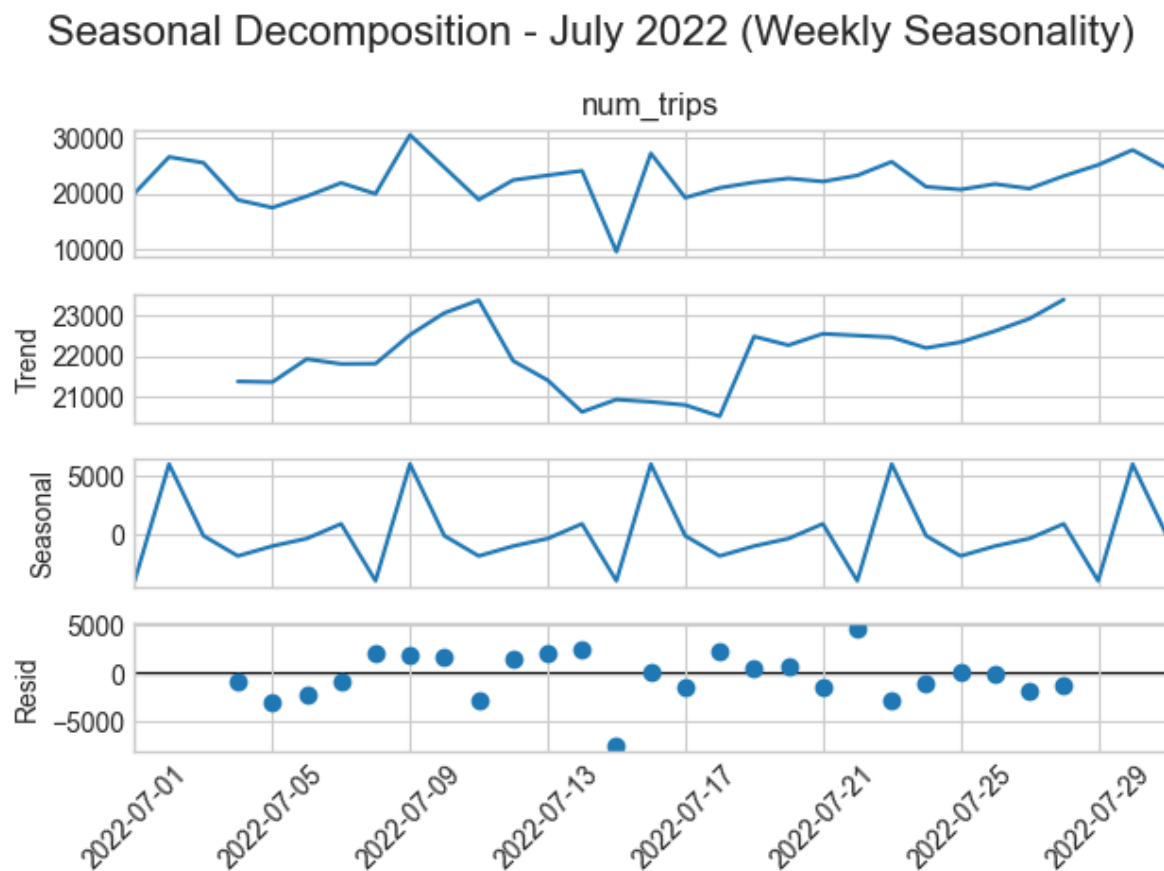
# do seasonal decomposition for the year 2022, month 7
daily_data_2022_7_indexed = daily_data_2022_7.copy()
daily_data_2022_7_indexed['started_date'] = pd.to_datetime(daily_data_2022_7_indexed['started_date'])
daily_data_2022_7_indexed.set_index('started_date', inplace=True)

result_2022_7 = seasonal_decompose(daily_data_2022_7_indexed['num_trips'], model='additive',

# plot the decomposition with proper date formatting
plt.figure(figsize=(14, 8))
result_2022_7.plot()
plt.suptitle('Seasonal Decomposition - July 2022 (Weekly Seasonality)', fontsize=16)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```

<Figure size 1400x800 with 0 Axes>



Combined Visualization

For a comprehensive analysis, plot both yearly and weekly seasonality components side by side. This approach allows you to compare long-term and short-term patterns and better understand the factors influencing bike usage.

```
# Use the indexed data for meaningful x-axis
daily_data_for_decomp = daily_data.copy()
daily_data_for_decomp['started_date'] = pd.to_datetime(daily_data_for_decomp['started_date'])
daily_data_for_decomp.set_index('started_date', inplace=True)

# Decompose with weekly seasonality
result_weekly = seasonal_decompose(daily_data_for_decomp['num_trips'], model='additive', per

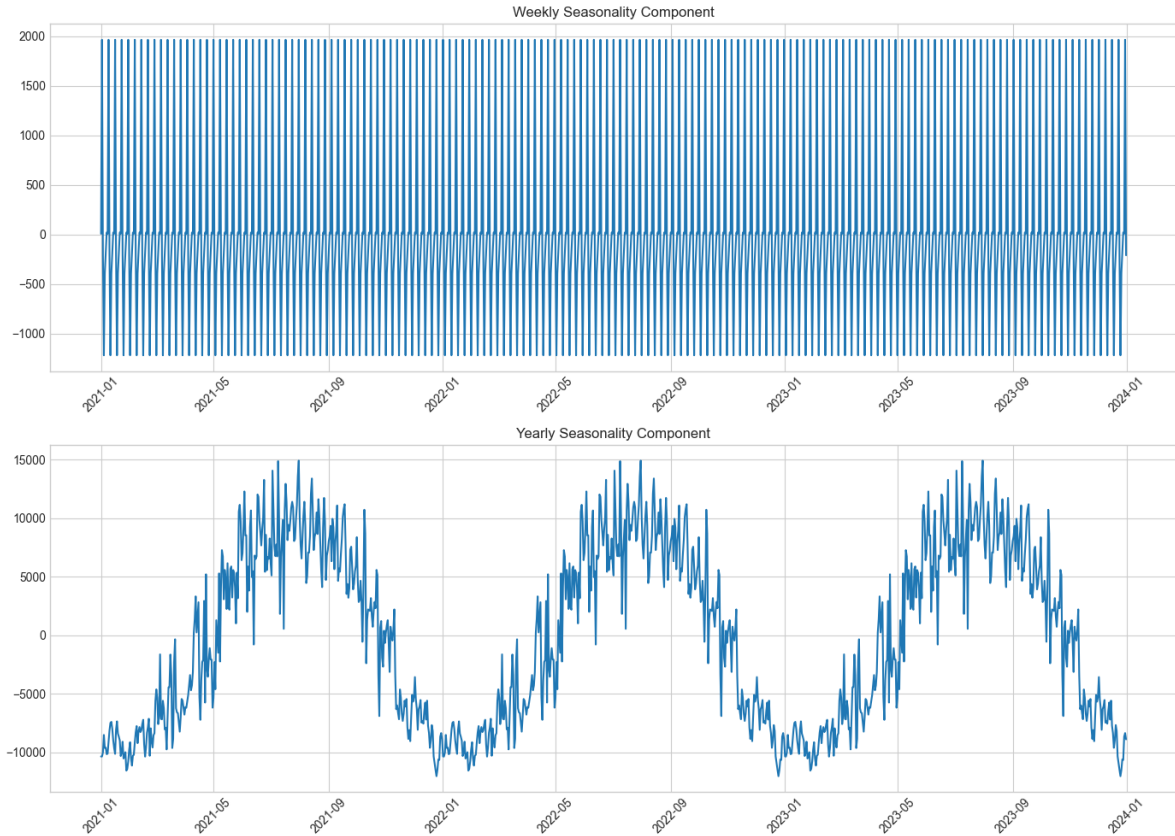
# Decompose with yearly seasonality
result_yearly = seasonal_decompose(daily_data_for_decomp['num_trips'], model='additive', per

# Plot both decompositions side by side with meaningful dates
fig, axes = plt.subplots(2, 1, figsize=(14, 10))

axes[0].plot(result_weekly.seasonal.index, result_weekly.seasonal)
axes[0].set_title('Weekly Seasonality Component')
axes[0].tick_params(axis='x', rotation=45)

axes[1].plot(result_yearly.seasonal.index, result_yearly.seasonal)
axes[1].set_title('Yearly Seasonality Component')
axes[1].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```



Takeaways:

- Use weekly seasonality to capture short-term, day-to-day variations.
- Use yearly seasonality to understand long-term trends and annual cycles.
- Visualizing both helps build more accurate models and provides deeper insights into ridership patterns.

Choosing Additive vs. Multiplicative Models

- **Additive Model:** The time series is expressed as the sum of the trend, seasonal, and residual components. This model is suitable when the magnitude of the seasonal fluctuations is approximately constant over time.

$$\text{Observed} = \text{Trend} + \text{Seasonal} + \text{Residual}$$

Additive model is suitable if seasonal fluctuations are constant over time.

- **Multiplicative Model:** The time series is expressed as the product of the trend, seasonal, and residual components. This model is appropriate when the seasonal fluctuations change proportionally with the level of the series, meaning the magnitude of the seasonal effects increases with the level of the time series.

$$\text{Observed} = \text{Trend} \times \text{Seasonal} \times \text{Residual}$$

Multiplicative Model is suitable if seasonal fluctuations change proportionally with the level of the time series.

By visually inspecting the decompositions above, it is evident that the additive model captures the characteristics of the Divvy daily trips data effectively.

Step 4: Lag Analysis (Lag Plots, ACF, PACF)

Lag analysis helps us understand how past values influence current values in a time series. This is key for choosing the right forecasting model.

- **Lag Plots:** Show the relationship between a value and its previous values (lags). Useful for spotting autocorrelation and outliers.
- **Autocorrelation Function (ACF):** Measures how strongly current values relate to past values at different lags. Guides the choice of MA (Moving Average) terms.
- **Partial Autocorrelation Function (PACF):** Shows the direct effect of a lag, removing the influence of shorter lags. Helps select AR (AutoRegressive) terms.

How to use these tools: 1. Start with lag plots to visually check for autocorrelation or patterns. 2. Use ACF to see which lags have strong correlations (for MA terms). 3. Use PACF to find which lags have direct effects (for AR terms).

Combining these plots gives a clear picture of time dependencies and helps you build better time series models.

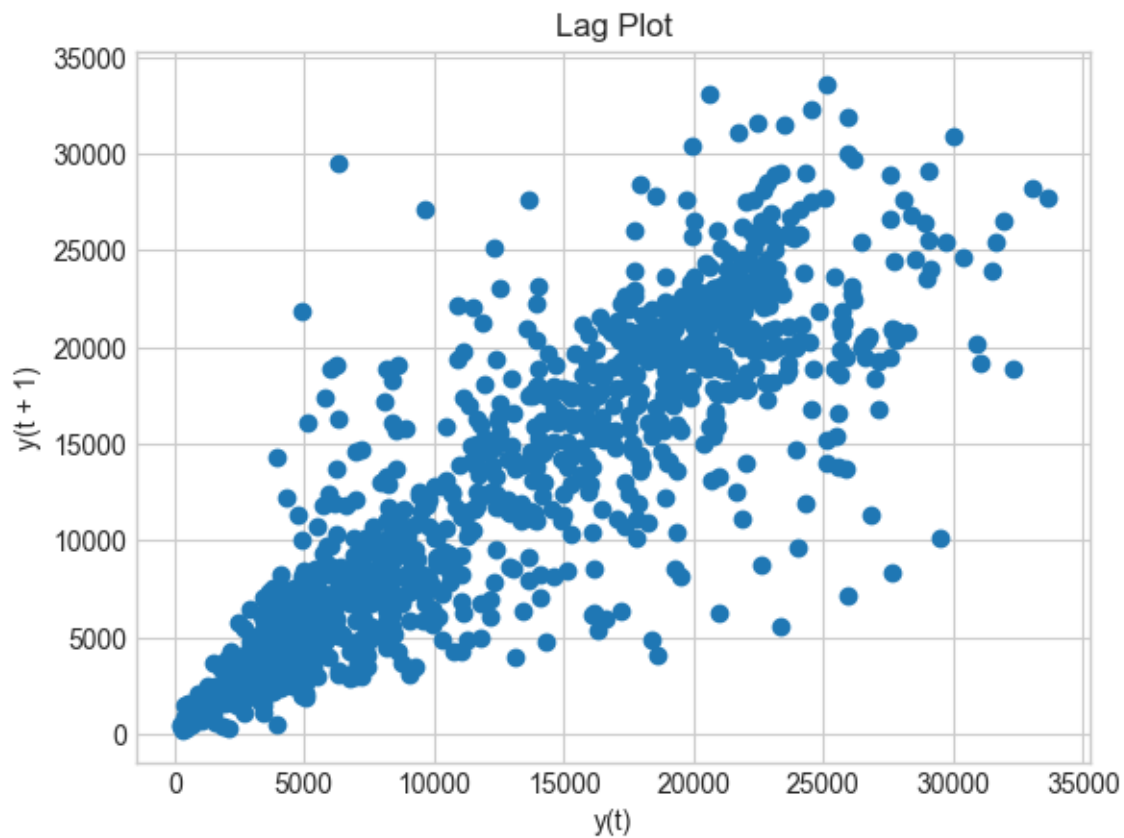
Lag Plots

```
from pandas.plotting import lag_plot

# Assuming your time series data is in a pandas Series called 'ts'
# If it's not, you can create one like this:
# ts = pd.Series(your_data, index=your_dates)

# Create the lag plot
lag_plot(daily_data['num_trips'], lag=1)
```

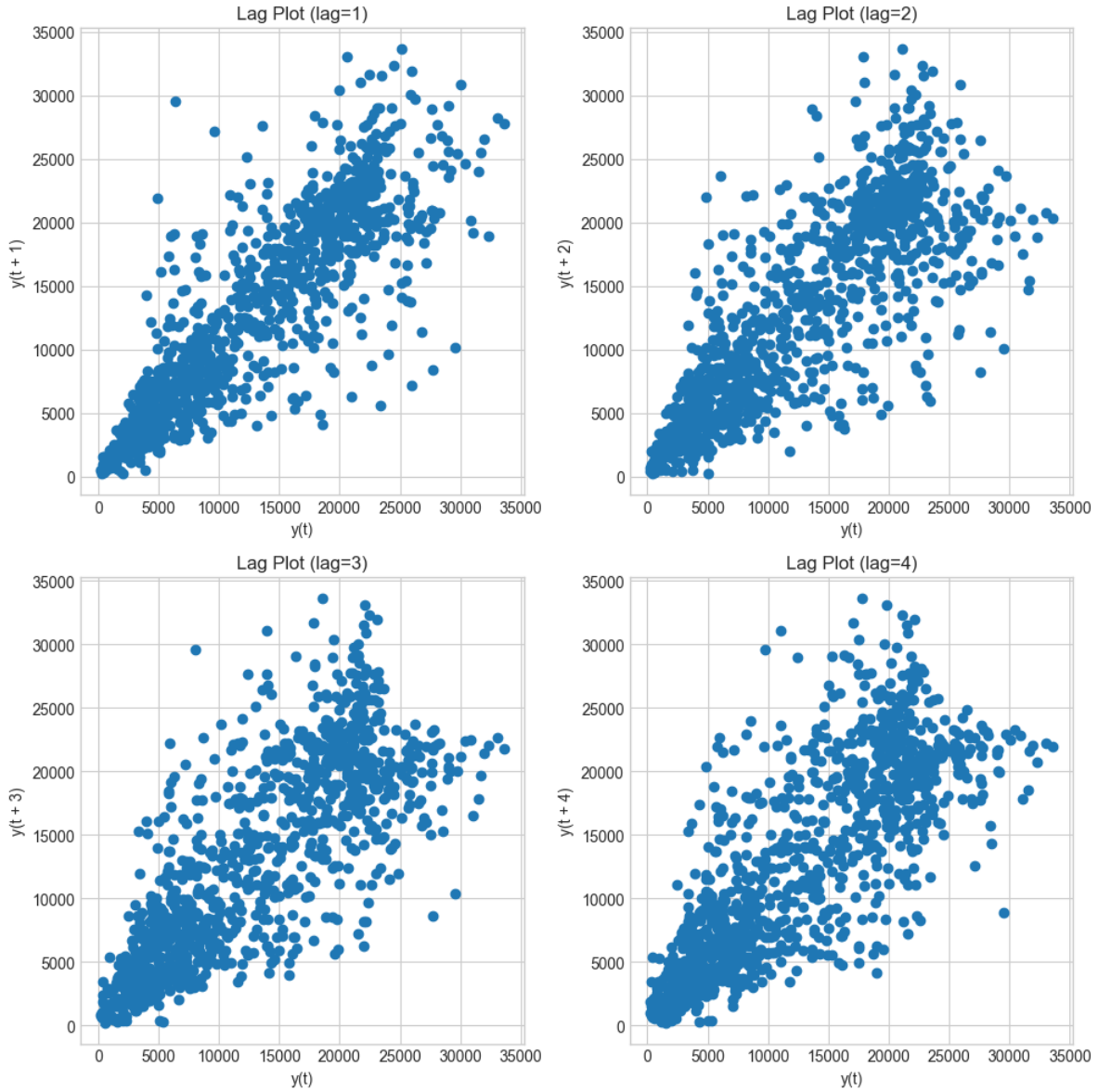
```
plt.title('Lag Plot')
plt.show()
```



This code creates a lag plot with a lag of 1. You can adjust the lag by changing the `lag` parameter.

To create multiple lag plots:

```
# create a 2x2 grid of lag plots for lags 1 to 4
fig, axes = plt.subplots(2, 2, figsize=(10, 10))
for i, ax in enumerate(axes.flatten()[4]):
    lag_plot(daily_data['num_trips'], lag=i+1, ax=ax)
    ax.set_title(f'Lag Plot (lag={i+1})')
plt.tight_layout()
plt.show()
```



Lag plots are valuable tools for time series analysis and can inform your modeling approach in several ways: 1. **Autocorrelation detection:** * If points cluster along the diagonal, it indicates positive autocorrelation. * Scattered points suggest little to no autocorrelation.

2. **Non-linearity assessment:**

- Curved patterns suggest non-linear relationships between lagged values.

3. **Outlier identification:**

- Points far from the main cluster may be outliers.

4. Model selection:

- Strong relationships at certain lags can suggest appropriate AR or MA terms for ARIMA models.
- Complex patterns might indicate the need for more sophisticated models (e.g., SARIMA, neural networks).

5. Determining differencing:

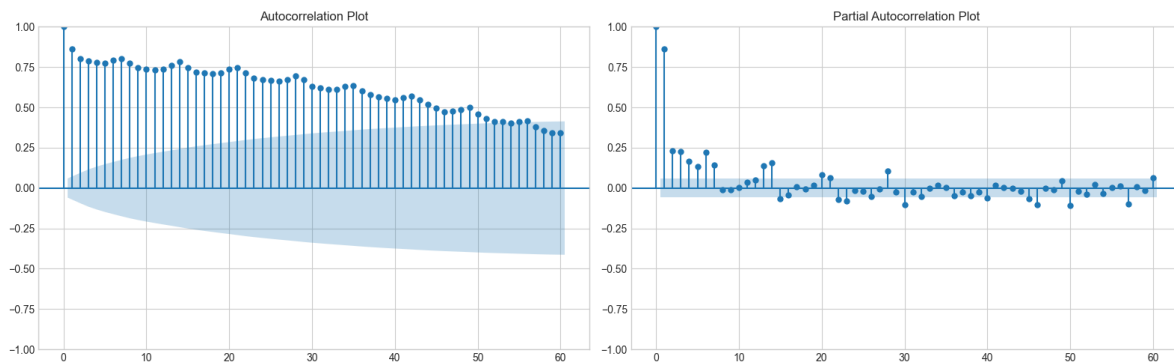
- If lag-1 plot shows strong linear relationship, differencing may be needed.

Plotting ACF and PACF

To generate autocorrelation (ACF) and partial autocorrelation (PACF) plots in Python, you can use the statsmodels library

```
# plot ACF and PACF
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 5))
plot_acf(daily_data['num_trips'], ax=ax1, lags=60, alpha=0.05)
ax1.set_title('Autocorrelation Plot')
plot_pacf(daily_data['num_trips'], ax=ax2, lags=60, alpha=0.05)
ax2.set_title('Partial Autocorrelation Plot')
plt.tight_layout()
plt.show()
```



Key points:

- Adjust the `lags` parameter to show more or fewer lags.
- The blue shaded area represents the confidence interval (default is 95%).
- Bars extending beyond this area are statistically significant at that confidence level.

Step 5: Checking for Stationarity

Many forecasting models, like ARIMA, require the time series to be stationary—meaning its mean and variance do not change over time. Seasonality can affect stationarity, so we often use Seasonal ARIMA (SARIMA) for seasonal data.

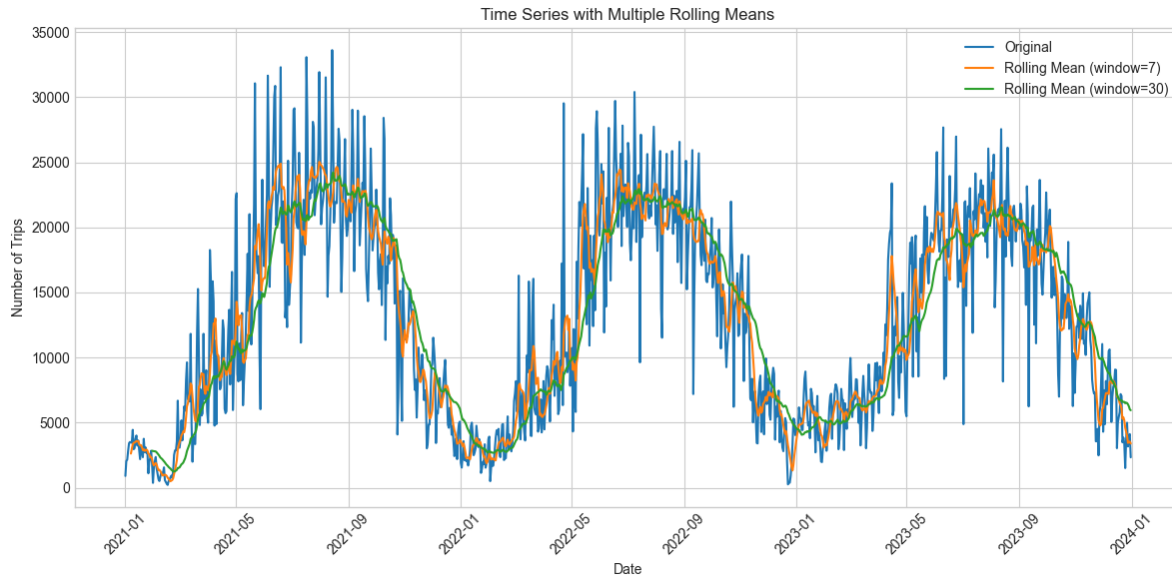
How to check stationarity: - **Rolling Means:** Plot moving averages to see if the mean stays roughly constant. - **Augmented Dickey-Fuller (ADF) Test:** A statistical test for stationarity. A low p-value (0.05) means the series is likely stationary.

If the series is not stationary, you may need to difference the data or apply transformations (like log). The ADF test is a standard way to check for stationarity before modeling.

```
window_sizes = [7, 30] # Adjust as needed
rolling_means = {size: daily_data['num_trips'].rolling(window=size).mean() for size in window_sizes}

# Ensure started_date is datetime for meaningful x-axis
daily_data['started_date'] = pd.to_datetime(daily_data['started_date'])

plt.figure(figsize=(12, 6))
plt.plot(daily_data['started_date'], daily_data['num_trips'], label='Original')
for size, mean in rolling_means.items():
    plt.plot(daily_data['started_date'], mean, label=f'Rolling Mean (window={size})')
plt.legend()
plt.title('Time Series with Multiple Rolling Means')
plt.xlabel('Date')
plt.ylabel('Number of Trips')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



The number of trips shows strong recurring seasonality, peaking in the summer and declining in the winter. Rolling means smooth out noise and highlight these cycles, making it easier to spot trends and changes in the average over time.

```
from statsmodels.tsa.stattools import adfuller

def adf_test(series):
    result = adfuller(series, autolag='AIC')
    print('ADF Statistic:', result[0])
    print('p-value:', result[1])
    print('Critical Values:')
    for key, value in result[4].items():
        print(f'    {key}: {value}')

# Perform ADF test
adf_test(daily_data['num_trips'])
```

```
ADF Statistic: -1.7573071952000077
p-value: 0.4018538499381927
Critical Values:
    1%: -3.4364647646486093
    5%: -2.864239892228526
    10%: -2.5682075189699822
```

How to interpret ADF test results:

- **ADF Statistic:** More negative values mean stronger evidence for stationarity.
- **p-value:**
 - If p-value ≤ 0.05 : The series is likely stationary.
 - If p-value > 0.05 : The series is likely non-stationary.
- **Critical Values:**
 - If the ADF statistic is less than the critical value, the series is stationary at that confidence level.

If the series is not stationary, try differencing (subtracting previous values) or applying a log transformation. The number of differences needed helps set the ‘d’ parameter in ARIMA models. Avoid over-differencing, as it can make the model unnecessarily complex.

```
print("Original Series:")
adf_test(daily_data['num_trips'])

print("\nFirst Difference:")
adf_test(daily_data['num_trips'].diff().dropna())

print("\nSecond Difference:")
adf_test(daily_data['num_trips'].diff().diff().dropna())
```

```
Original Series:
ADF Statistic: -1.7573071952000077
p-value: 0.4018538499381927
Critical Values:
  1%: -3.4364647646486093
  5%: -2.864239892228526
 10%: -2.5682075189699822
```

```
First Difference:
ADF Statistic: -8.6635824523991
p-value: 4.750032021062828e-14
Critical Values:
  1%: -3.4364647646486093
  5%: -2.864239892228526
 10%: -2.5682075189699822
```

```
Second Difference:
ADF Statistic: -15.46298976175681
p-value: 2.690625105696648e-28
Critical Values:
```

```
1%: -3.4364762217105844
5%: -2.8642449457774135
10%: -2.5682102104987337
```

Key points:

- If the series is non-stationary, differencing is often needed before modeling.
- The number of differences required informs the 'd' parameter in ARIMA.
- Over-differencing can add noise and should be avoided.

In this case, the first difference makes the series stationary (p-value < 0.05). A second difference is unnecessary and may complicate the model.

Log transformation can help stabilize variance in a time series, making it easier to model and forecast. Let's apply a log transformation to the number of trips and check for stationarity again.

```
# perform log transformation on the number of trips, add it to the daily_data
daily_data['num_trips_log'] = np.log(daily_data['num_trips'])

# perform ADF test on the log transformed series
print("="*50)
print("ADF TEST ON LOG TRANSFORMED SERIES")
print("="*50)
print("Log Transformed Series:")
adf_test(daily_data['num_trips_log'])

print("\nFirst Difference of Log Transformed Series:")
adf_test(daily_data['num_trips_log'].diff().dropna())
```

```
=====
ADF TEST ON LOG TRANSFORMED SERIES
=====
Log Transformed Series:
ADF Statistic: -2.2616271881013565
p-value: 0.18464268146685497
Critical Values:
  1%: -3.4364533503600962
  5%: -2.864234857527328
 10%: -2.568204837482531
```

```
First Difference of Log Transformed Series:
```

ADF Statistic: -9.728219800968347

p-value: 9.152102299828489e-17

Critical Values:

1%: -3.4364533503600962

5%: -2.864234857527328

10%: -2.568204837482531

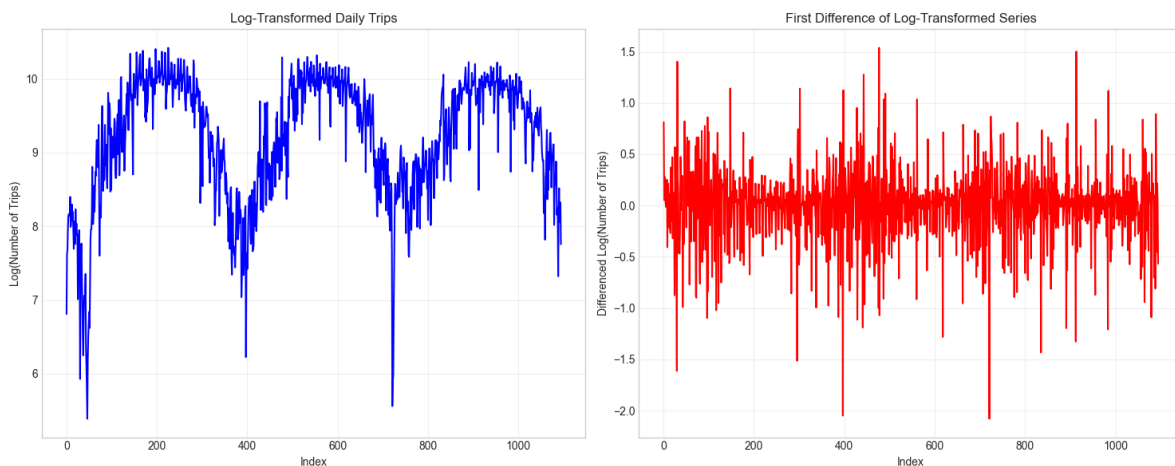
The first difference of the log-transformed series has a p-value < 0.05 , indicating it is stationary and ready for modeling.

```
# Plot log-transformed series and its first difference side by side
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

# Plot log-transformed series
daily_data['num_trips_log'].plot(ax=ax1, title='Log-Transformed Daily Trips', color='blue')
ax1.set_xlabel('Index')
ax1.set_ylabel('Log(Number of Trips)')
ax1.grid(True, alpha=0.3)

# Plot first difference of log series
daily_data['num_trips_log'].diff().plot(ax=ax2, title='First Difference of Log-Transformed Series')
ax2.set_xlabel('Index')
ax2.set_ylabel('Differenced Log(Number of Trips)')
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



Since we have already confirmed the series is **stationary**, we can proceed with **ARIMA**, a classic baseline model for univariate time series forecasting. ARIMA models temporal dependence using only past values of the series (i.e., without incorporating external covariates such as weather or holidays).

Time Series Prediction with ARIMA

ARIMA (AutoRegressive Integrated Moving Average) combines three components:

- **AR (AutoRegressive)**: Uses past values to predict future values.
- **I (Integrated)**: Represents the differencing order (here, minimal or none because the series is already stationary).
- **MA (Moving Average)**: Models the relationship between an observation and residual errors from previous observations.

Next, we fit an ARIMA model to the **(stationary) log-differenced daily trips** and evaluate its forecasting performance.

```
# import mean_absolute_error
from sklearn.metrics import mean_absolute_error
import statsmodels.api as sm
X_df = (
    daily_data[['started_date', 'num_trips_log']]
    .set_index('started_date')
    .asfreq('D') # Explicitly set daily frequency
    ['num_trips_log']
    .diff()
    .dropna()
)

# Fit ARIMA model (p=1, d=1, q=1)
arima_model = sm.tsa.ARIMA(X_df, order=(1, 1, 1))
arima_results = arima_model.fit()

# Generate predictions
predictions = arima_results.predict(start=X_df.index[0], end=X_df.index[-1], typ='levels')

# Calculate Mean Absolute Error (MAE)
mae = mean_absolute_error(X_df, predictions)
print(f'Mean Absolute Error: {mae}')

# Print model summary
print(arima_results.summary())
```

```
# Apply ARIMA on the differenced series
p = 1
d = 1
q = 1

model = sm.tsa.ARIMA(X_df, order=(p, d, q))
results = model.fit()
mae = np.mean(np.abs(results.resid))
print(f'Mean Absolute Error: {mae}')
# Print summary of the model
print(results.summary())

# Optional: plot the residuals to check for stationarity and autocorrelation
results.plot_diagnostics(figsize=(10, 6));
```

Mean Absolute Error: 0.2598103296981036

SARIMAX Results

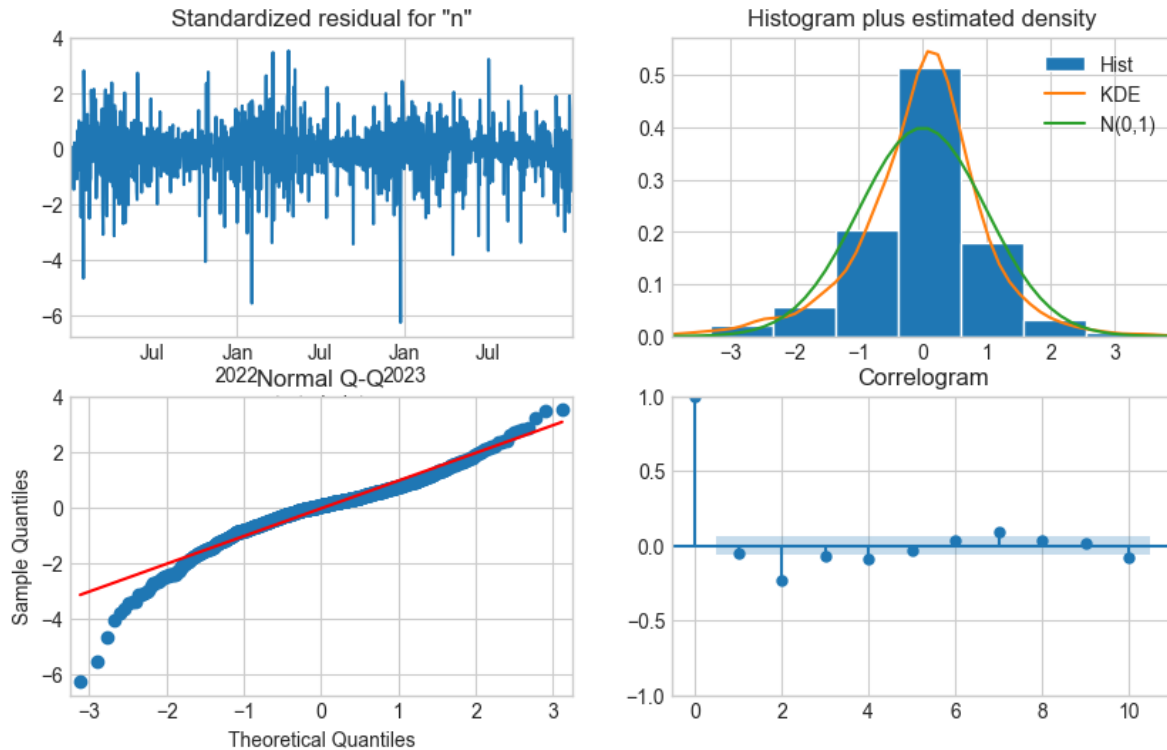
```
=====
Dep. Variable:          num_trips_log    No. Observations:          1094
Model:                  ARIMA(1, 1, 1)    Log Likelihood              -455.573
Date:                   Thu, 04 Sep 2025    AIC                         917.147
Time:                   11:56:52           BIC                         932.137
Sample:                 01-02-2021         HQIC                        922.819
                   - 12-31-2023
Covariance Type:                opg
=====
```

	coef	std err	z	P> z	[0.025	0.975]
ar.L1	-0.2400	0.023	-10.433	0.000	-0.285	-0.195
ma.L1	-0.9992	0.021	-48.729	0.000	-1.039	-0.959
sigma2	0.1339	0.004	37.552	0.000	0.127	0.141

```
=====
Ljung-Box (L1) (Q):                2.99    Jarque-Bera (JB):                772.62
Prob(Q):                           0.08    Prob(JB):                        0.00
Heteroskedasticity (H):             0.80    Skew:                            -0.84
Prob(H) (two-sided):               0.04    Kurtosis:                        6.76
=====
```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).



Time Series Prediction with `auto_arima`

`auto_arima` automates the process of finding the best ARIMA parameters for a univariate time series. It tests different combinations of ARIMA parameters and selects the model with the best fit based on statistical criteria. This approach is useful for quickly identifying a reasonable ARIMA model, but it still does not address seasonality or external variables unless explicitly enabled (e.g., with `seasonal=True` for SARIMA).

- **Key Point:** `auto_arima` is still univariate unless you use exogenous variables (not shown here).
- **Limitation:** For Divvy data, even the best ARIMA model may not capture all patterns due to strong seasonality and external influences.

```
from statsmodel_auto_arima import auto_sarimax
# First, import or define the auto_sarimax function (if not already done)
# You can either copy the function definition here or import it from a separate file

# Fit auto_sarimax with seasonal settings
model = auto_sarimax(X_df,
                     start_p=1, start_q=1,
```

```

        max_p=5, max_q=5,
        seasonal=True, # Enable seasonal ARIMA
        m=7, # Set season length (e.g., 7 for weekly seasonality)
        start_P=0, start_Q=0,
        max_P=3, max_Q=3,
        trace=True,
        error_action='ignore',
        suppress_warnings=True,
        stepwise=True)

mae = np.mean(np.abs(model.resid))
print(f'Mean Absolute Error: {mae}')
# Summary of the best seasonal ARIMA model
print(model.summary())

# Optional: Plot diagnostics of the best model
model.plot_diagnostics(figsize=(10, 6));

```

Determined d=0, D=0

```

Trying SARIMAX(1, 0, 1)x(0, 0, 0, 7)
New best model: SARIMAX(1, 0, 1)x(0, 0, 0, 7) - AIC: 772.184
Trying SARIMAX(1, 0, 1)x(0, 0, 3, 7)
New best model: SARIMAX(1, 0, 1)x(0, 0, 3, 7) - AIC: 758.099
Trying SARIMAX(1, 0, 1)x(3, 0, 0, 7)
New best model: SARIMAX(1, 0, 1)x(3, 0, 0, 7) - AIC: 752.941
Trying SARIMAX(1, 0, 1)x(3, 0, 3, 7)
New best model: SARIMAX(1, 0, 1)x(3, 0, 3, 7) - AIC: 742.041
Trying SARIMAX(1, 0, 2)x(0, 0, 0, 7)
Trying SARIMAX(1, 0, 2)x(0, 0, 3, 7)
Trying SARIMAX(1, 0, 2)x(3, 0, 0, 7)
Trying SARIMAX(1, 0, 2)x(3, 0, 3, 7)
Trying SARIMAX(1, 0, 5)x(0, 0, 0, 7)
Trying SARIMAX(1, 0, 5)x(0, 0, 3, 7)
Trying SARIMAX(1, 0, 5)x(3, 0, 0, 7)
Trying SARIMAX(1, 0, 5)x(3, 0, 3, 7)
New best model: SARIMAX(1, 0, 5)x(3, 0, 3, 7) - AIC: 738.071
Trying SARIMAX(2, 0, 1)x(0, 0, 0, 7)
Trying SARIMAX(2, 0, 1)x(0, 0, 3, 7)
Trying SARIMAX(2, 0, 1)x(3, 0, 0, 7)
Trying SARIMAX(2, 0, 1)x(3, 0, 3, 7)
Trying SARIMAX(2, 0, 2)x(0, 0, 0, 7)
Trying SARIMAX(2, 0, 2)x(0, 0, 3, 7)

```

```

Trying SARIMAX(2, 0, 2)x(3, 0, 0, 7)
Trying SARIMAX(2, 0, 2)x(3, 0, 3, 7)
Trying SARIMAX(2, 0, 5)x(0, 0, 0, 7)
Trying SARIMAX(2, 0, 5)x(0, 0, 3, 7)
Trying SARIMAX(2, 0, 5)x(3, 0, 0, 7)
Trying SARIMAX(2, 0, 5)x(3, 0, 3, 7)
Trying SARIMAX(5, 0, 1)x(0, 0, 0, 7)
Trying SARIMAX(5, 0, 1)x(0, 0, 3, 7)
Trying SARIMAX(5, 0, 1)x(3, 0, 0, 7)
Trying SARIMAX(5, 0, 1)x(3, 0, 3, 7)
New best model: SARIMAX(5, 0, 1)x(3, 0, 3, 7) - AIC: 730.675
Trying SARIMAX(5, 0, 2)x(0, 0, 0, 7)
Trying SARIMAX(5, 0, 2)x(0, 0, 3, 7)
Trying SARIMAX(5, 0, 2)x(3, 0, 0, 7)
Trying SARIMAX(5, 0, 2)x(3, 0, 3, 7)
Trying SARIMAX(5, 0, 5)x(0, 0, 0, 7)
Trying SARIMAX(5, 0, 5)x(0, 0, 3, 7)
Trying SARIMAX(5, 0, 5)x(3, 0, 0, 7)
Trying SARIMAX(5, 0, 5)x(3, 0, 3, 7)

```

```

Best model: SARIMAX(5, 0, 1)x(3, 0, 3, 7)
AIC: 730.675
Mean Absolute Error: 0.2381487270418346

```

SARIMAX Results

```

=====
Dep. Variable:          num_trips_log    No. Observations:
Model:                SARIMAX(5, 0, 1)x(3, 0, [1, 2, 3], 7)    Log Likelihood        -35
Date:                  Thu, 04 Sep 2025    AIC                    730
Time:                  11:58:46            BIC                    800
Sample:                01-02-2021        HQIC                   75
                        - 12-31-2023

```

```

Covariance Type:                opg

```

```

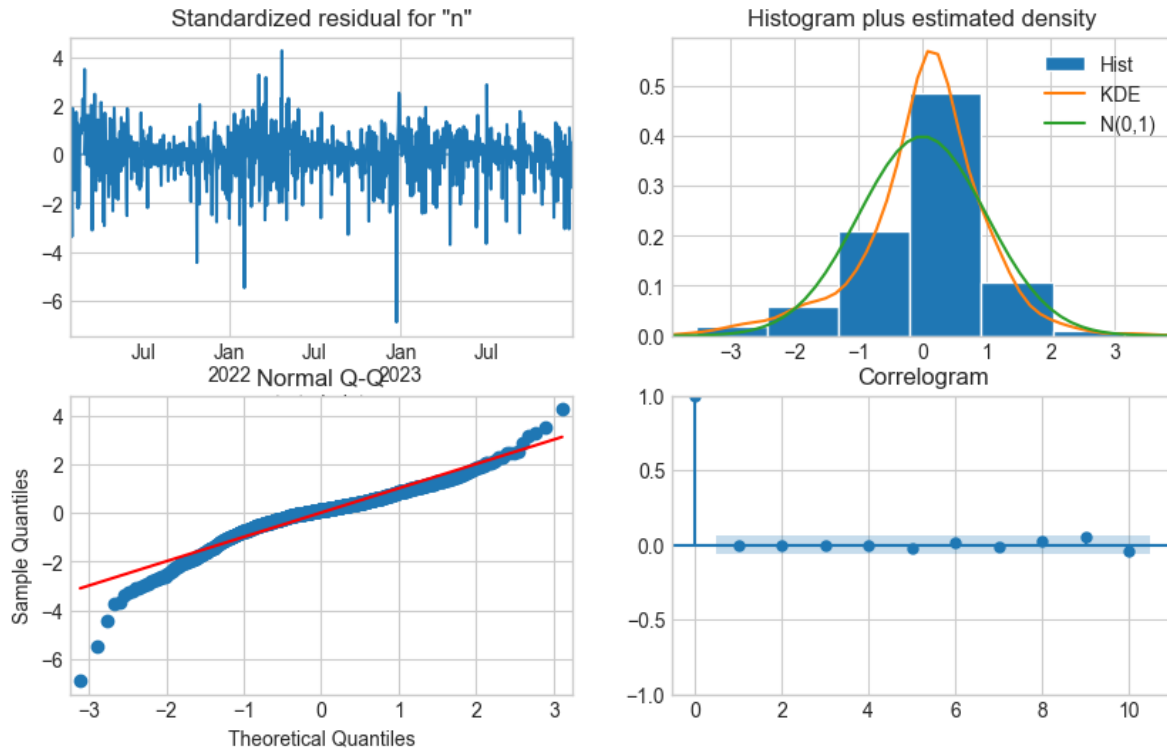
=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
intercept    -2.99e-05     0.000     -0.063     0.949     -0.001     0.001
ar.L1         0.5228      0.037     14.195     0.000      0.451     0.595
ar.L2         0.0198      0.035      0.572     0.567     -0.048     0.088
ar.L3         0.0922      0.036      2.572     0.010      0.022     0.162
ar.L4        -0.0232      0.037     -0.629     0.530     -0.095     0.049
ar.L5         0.0612      0.031      1.953     0.051     -0.000     0.123
ma.L1        -0.9286      0.031    -30.153     0.000     -0.989    -0.868
ar.S.L7       -0.1743      0.119     -1.468     0.142     -0.407     0.058

```

ar.S.L14	0.3498	0.079	4.444	0.000	0.195	0.504
ar.S.L21	0.5334	0.073	7.321	0.000	0.391	0.676
ma.S.L7	0.2647	0.123	2.146	0.032	0.023	0.506
ma.S.L14	-0.2289	0.083	-2.753	0.006	-0.392	-0.066
ma.S.L21	-0.4962	0.074	-6.673	0.000	-0.642	-0.350
sigma2	0.1123	0.003	35.700	0.000	0.106	0.118
=====						
Ljung-Box (L1) (Q):	0.05		Jarque-Bera (JB):	1028.05		
Prob(Q):	0.83		Prob(JB):	0.00		
Heteroskedasticity (H):	0.83		Skew:	-0.96		
Prob(H) (two-sided):	0.08		Kurtosis:	7.41		
=====						

Warnings:

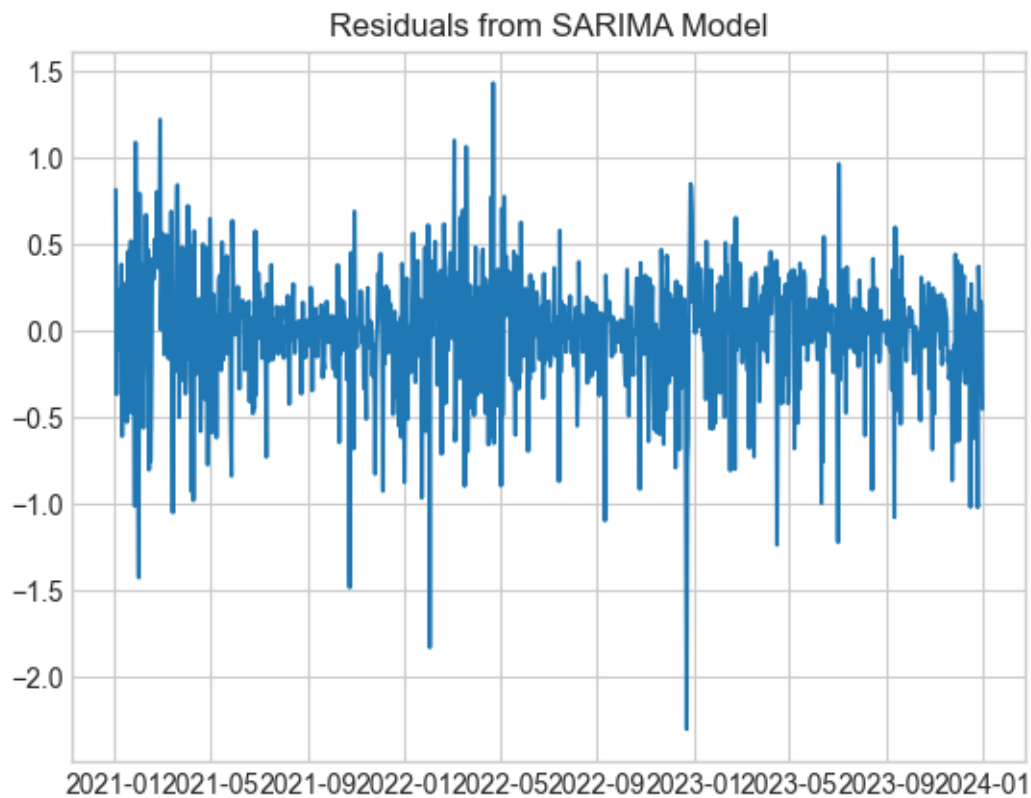
[1] Covariance matrix calculated using the outer product of gradients (complex-step).



After fitting an ARIMA/auto_arma model, we often examine the residuals (errors) to check for patterns in volatility.

```
# Extract residuals from the ARIMA model
residuals =model.resid
mae = np.mean(np.abs(residuals))
print(f'Mean Absolute Error: {mae}')
# Plot residuals to check for heteroscedasticity
plt.plot(residuals)
plt.title('Residuals from ARIMA Model')
plt.show();
```

Mean Absolute Error: 0.2381487270418346



Time Series Prediction with ARCH/GARCH Models

If the variance of the residuals changes over time (heteroskedasticity), ARCH (Autoregressive Conditional Heteroskedasticity) and GARCH (Generalized ARCH) models can be used to model and forecast this volatility. - **ARCH Model:** Captures time-varying volatility by modeling the variance of the current error as a function of past squared errors. - **GARCH**

Model: Extends ARCH by also including past forecasted variances, providing a more flexible approach to modeling volatility. - **Key Point:** Both ARCH and GARCH are univariate models applied to the residuals of a single time series. They do not use external variables. - **Limitation:** While these models can improve forecasts of uncertainty, they do not address the underlying seasonality or external drivers of bike usage.

```
from arch import arch_model

# Fit an ARCH model to the residuals
fitted_arch_model = arch_model(residuals, vol='ARCH', p=1) # p is the order of the lag for ARCH
arch_result = fitted_arch_model.fit()

mae = np.mean(np.abs(arch_result.resid))
print(f'Mean Absolute Error: {mae}')
# Summary of the ARCH model
print(arch_result.summary())

# Plot standardized residuals and conditional volatility
arch_result.plot(annualize='D') # 'D' assumes daily data

# If you want to make predictions of volatility
forecast = arch_result.forecast(horizon=5) # Horizon specifies the number of steps to forecast
print(forecast.variance[-1:]) # Print the forecasted variance
```

```
Iteration:      1,   Func. Count:      5,   Neg. LLF: 12799.864210635293
Iteration:      2,   Func. Count:     14,   Neg. LLF: 1218.6383322496313
Iteration:      3,   Func. Count:     22,   Neg. LLF: 404.5066417721329
Iteration:      4,   Func. Count:     28,   Neg. LLF: 339.01211006383664
Iteration:      5,   Func. Count:     32,   Neg. LLF: 339.0121089325292
Iteration:      6,   Func. Count:     35,   Neg. LLF: 339.0121089325705
```

Optimization terminated successfully (Exit mode 0)

Current function value: 339.0121089325292

Iterations: 6

Function evaluations: 35

Gradient evaluations: 6

Mean Absolute Error: 0.23948480073888032

Constant Mean - ARCH Model Results

```
=====
Dep. Variable:          None    R-squared:          0.000
Mean Model:             Constant Mean    Adj. R-squared:      0.000
Vol Model:              ARCH    Log-Likelihood:     -339.012
Distribution:           Normal    AIC:              684.024
```

```

Method:          Maximum Likelihood    BIC:          699.017
                                   No. Observations:    1094
Date:            Thu, Sep 04 2025    Df Residuals:    1093
Time:            12:00:09    Df Model:          1

```

Mean Model

```

=====
              coef      std err          t      P>|t|      95.0% Conf. Int.
-----
mu          -8.5930e-03  9.667e-03     -0.889     0.374  [-2.754e-02,1.035e-02]

```

Volatility Model

```

=====
              coef      std err          t      P>|t|      95.0% Conf. Int.
-----
omega         0.0861  8.541e-03     10.078  6.908e-24  [6.934e-02, 0.103]
alpha[1]       0.2816  8.578e-02      3.283  1.027e-03  [ 0.113, 0.450]
=====

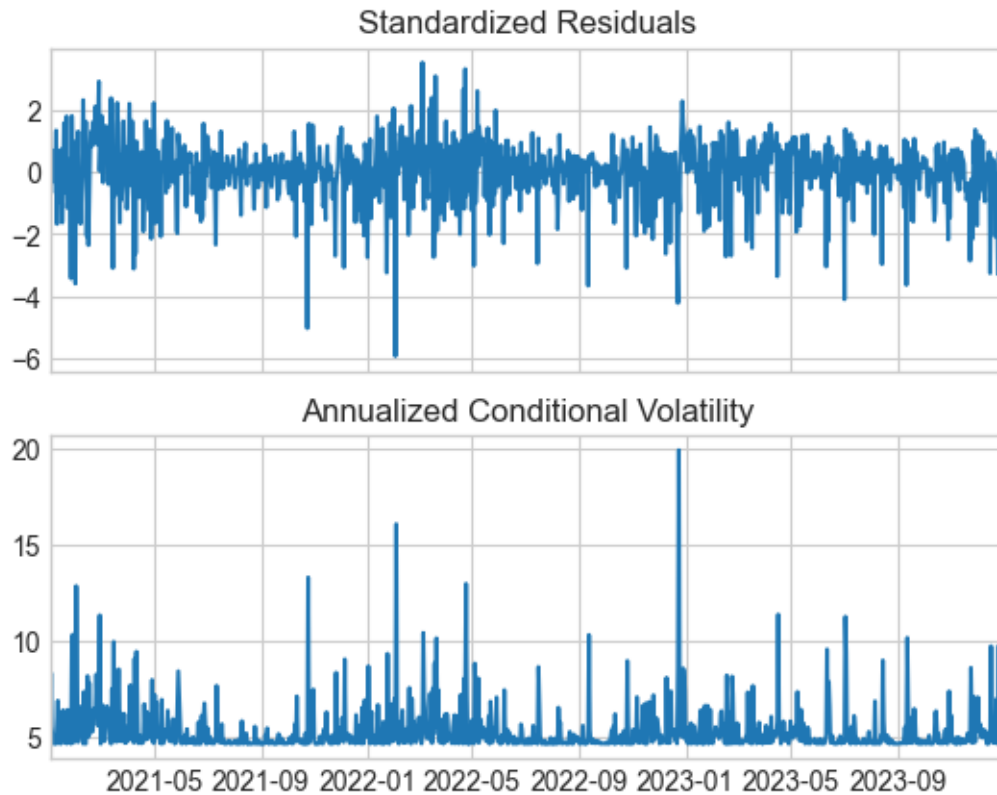
```

Covariance estimator: robust

```

              h.1      h.2      h.3      h.4      h.5
started_date
2023-12-31    0.142242  0.126134  0.121598  0.12032  0.119961

```



```
# Fit a GARCH model
garch_model = arch_model(residuals, vol='Garch', p=1, q=1)
garch_result = garch_model.fit()

mae = np.mean(np.abs(garch_result.resid))
print(f'Mean Absolute Error: {mae}')
# Summary of the GARCH model
print(garch_result.summary())

# Example for GARCH(1, 1)
garch_model = arch_model(residuals, vol='Garch', p=1, q=1)
garch_result = garch_model.fit()
garch_result.plot(annualize='D');
```

Iteration:	1,	Func. Count:	6,	Neg. LLF:	158142.3829703192
Iteration:	2,	Func. Count:	16,	Neg. LLF:	284133.60780683364
Iteration:	3,	Func. Count:	23,	Neg. LLF:	382.53171203951763
Iteration:	4,	Func. Count:	31,	Neg. LLF:	410.3379451333795

```

Iteration:      5,   Func. Count:      38,   Neg. LLF: 382.1101362675484
Iteration:      6,   Func. Count:      44,   Neg. LLF: 325.50854783221797
Iteration:      7,   Func. Count:      50,   Neg. LLF: 325.4327403456733
Iteration:      8,   Func. Count:      55,   Neg. LLF: 325.43059204089394
Iteration:      9,   Func. Count:      60,   Neg. LLF: 325.4305421268284
Iteration:     10,   Func. Count:      65,   Neg. LLF: 325.4305275048631
Iteration:     11,   Func. Count:      69,   Neg. LLF: 325.4305275048222

```

Optimization terminated successfully (Exit mode 0)

Current function value: 325.4305275048631

Iterations: 11

Function evaluations: 69

Gradient evaluations: 11

Mean Absolute Error: 0.2417108485240102

Constant Mean - GARCH Model Results

```

=====
Dep. Variable:          None    R-squared:                0.000
Mean Model:             Constant Mean    Adj. R-squared:          0.000
Vol Model:              GARCH    Log-Likelihood:         -325.431
Distribution:           Normal    AIC:                    658.861
Method:                 Maximum Likelihood    BIC:                    678.851
                                           No. Observations:      1094
Date:                   Thu, Sep 04 2025    Df Residuals:           1093
Time:                   12:00:12    Df Model:                1
=====

```

Mean Model

```

=====
              coef      std err          t      P>|t|      95.0% Conf. Int.
-----
mu          -0.0201   9.502e-03     -2.116   3.436e-02  [-3.873e-02,-1.481e-03]
=====

```

Volatility Model

```

=====
              coef      std err          t      P>|t|      95.0% Conf. Int.
-----
omega        0.0185   1.103e-02      1.679   9.324e-02  [-3.104e-03,4.014e-02]
alpha[1]     0.1838   8.509e-02      2.160   3.080e-02  [1.700e-02, 0.351]
beta[1]      0.6693    0.142         4.717   2.388e-06  [ 0.391, 0.947]
=====

```

Covariance estimator: robust

```

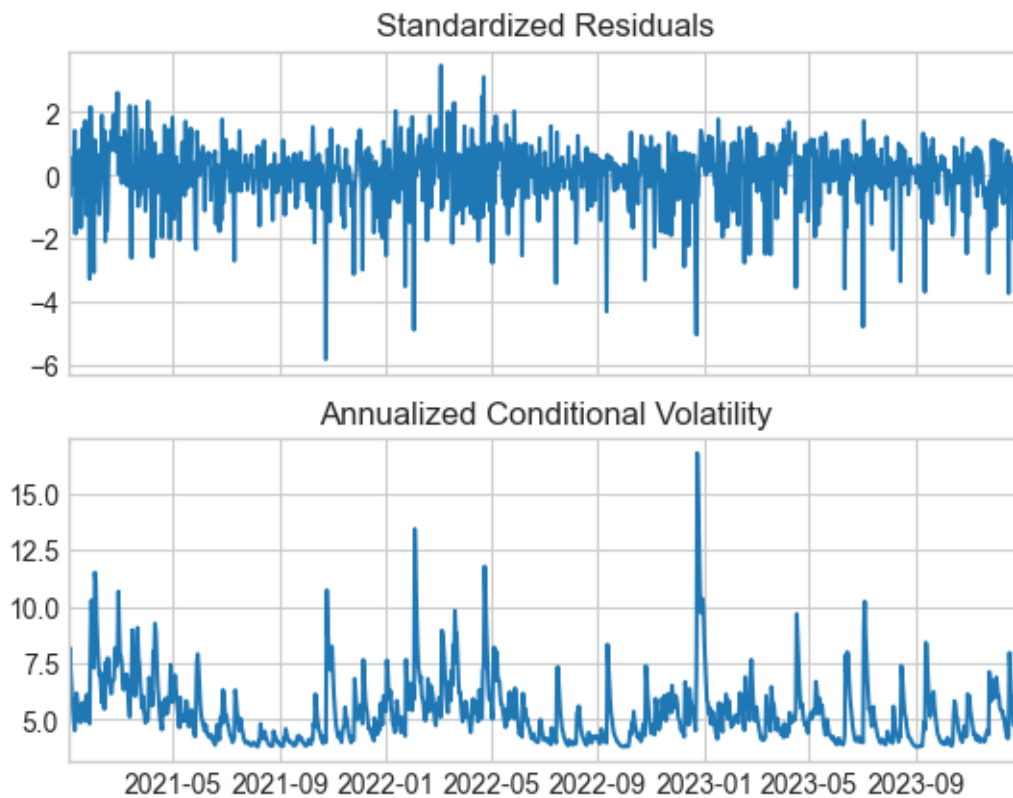
Iteration:      1,   Func. Count:      6,   Neg. LLF: 158142.3829703192
Iteration:      2,   Func. Count:     16,   Neg. LLF: 284133.60780683364
Iteration:      3,   Func. Count:     23,   Neg. LLF: 382.53171203951763
Iteration:      4,   Func. Count:     31,   Neg. LLF: 410.3379451333795
Iteration:      5,   Func. Count:     38,   Neg. LLF: 382.1101362675484

```

```

Iteration:      6,   Func. Count:    44,   Neg. LLF: 325.50854783221797
Iteration:      7,   Func. Count:    50,   Neg. LLF: 325.4327403456733
Iteration:      8,   Func. Count:    55,   Neg. LLF: 325.43059204089394
Iteration:      9,   Func. Count:    60,   Neg. LLF: 325.4305421268284
Iteration:     10,   Func. Count:    65,   Neg. LLF: 325.4305275048631
Iteration:     11,   Func. Count:    69,   Neg. LLF: 325.4305275048222
Optimization terminated successfully   (Exit mode 0)
      Current function value: 325.4305275048631
      Iterations: 11
      Function evaluations: 69
      Gradient evaluations: 11

```



Summary and Next Steps

In this notebook, we walked through a comprehensive workflow for time series modeling using Divvy bikeshare data.

Key Takeaways

Univariate models such as ARIMA/auto_arima and ARCH/GARCH are valuable for capturing patterns (and, for GARCH, volatility) in a single time series. However, for complex real-world data like Divvy bike trips, these models have important limitations: - **Persistent seasonality** (e.g., weekly and annual cycles) that is difficult to fully remove - **Non-stationarity** that may persist despite transformations - **External drivers** (e.g., weather, holidays, special events) that strongly affect demand but are not captured in univariate models

To produce more accurate and actionable forecasts, we need to move beyond univariate models and incorporate additional features. This sets the stage for multivariate modeling, where we use multiple variables to better capture the drivers of bike usage.

Next Steps

- **Explore additional external factors** and advanced modeling techniques to further enhance predictions.
- **Apply multivariate time series models** (such as XGBoost) to incorporate multiple features (e.g., temperature, holidays, lagged values) and better capture the drivers of bike usage.

By moving beyond univariate models, you'll be able to build more robust forecasts and gain deeper insights into the factors influencing Divvy ridership. This approach will help you deliver more accurate, actionable, and meaningful results for real-world decision making.